Outline

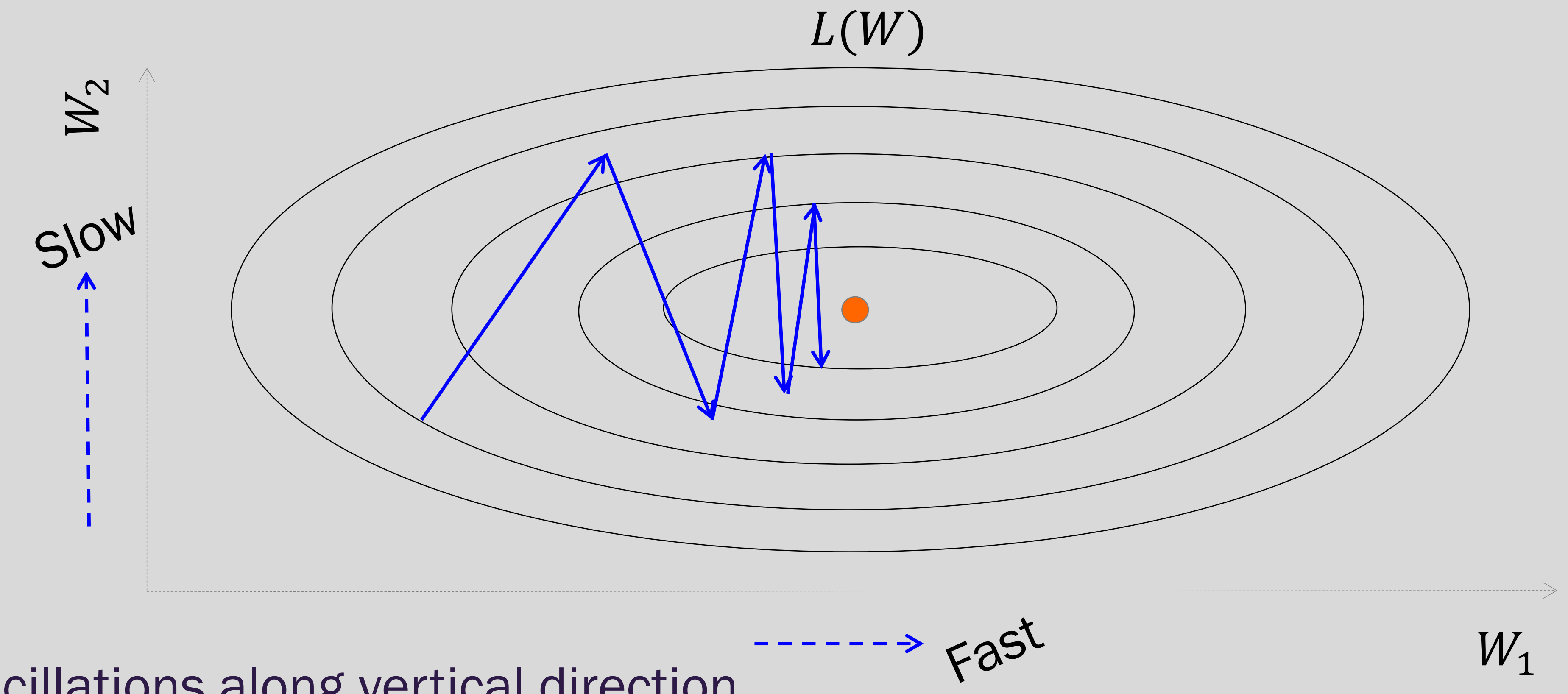
Optimization

- Challenges in Optimization
- Momentum
- **Adaptive Learning Rate**
- Parameter Initialization
- Batch Normalization

Regularization of NN

- Norm Penalties
- Early Stopping
- Data Augmentation
- Sparse Representation
- Dropout

Adaptive Learning Rates

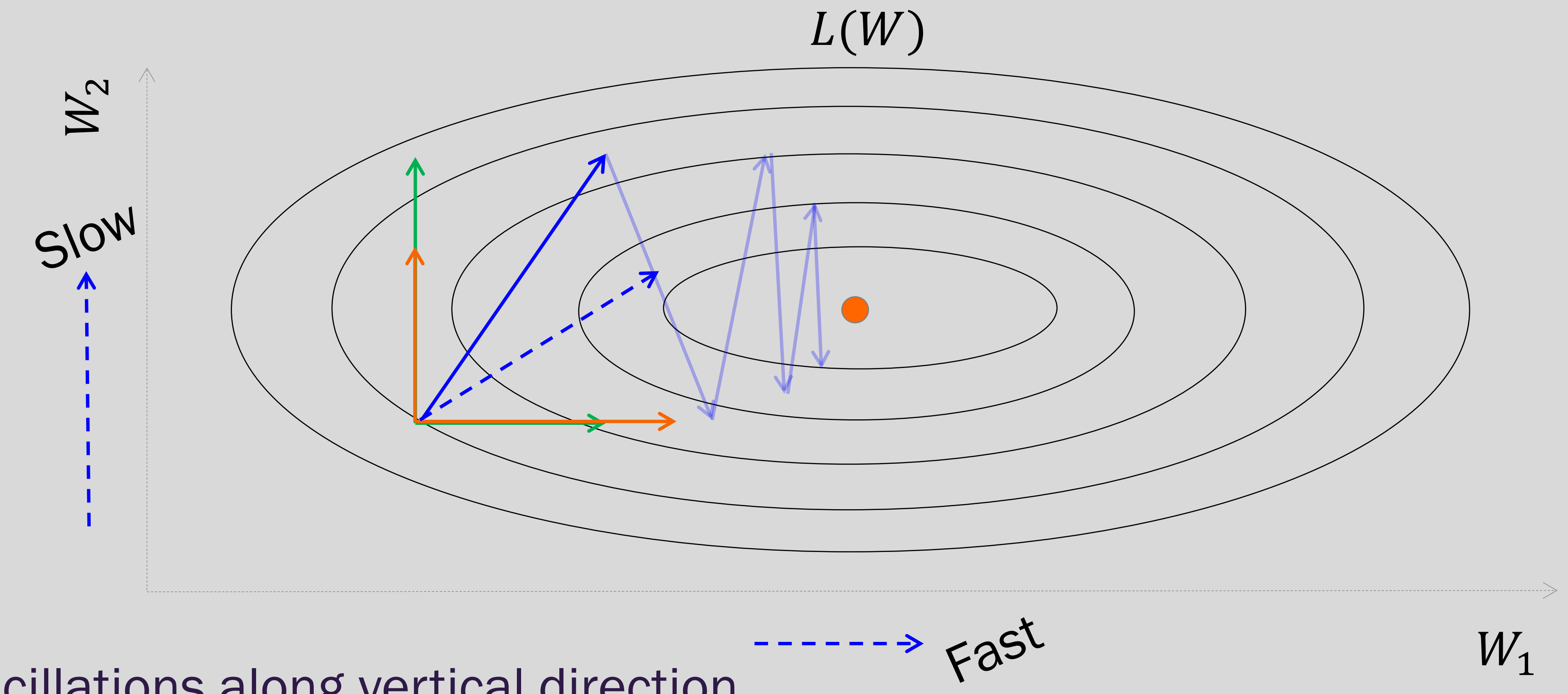


Oscillations along vertical direction

- Learning must be slower along parameter 2

Use a different learning rate for each parameter?

Adaptive Learning Rates

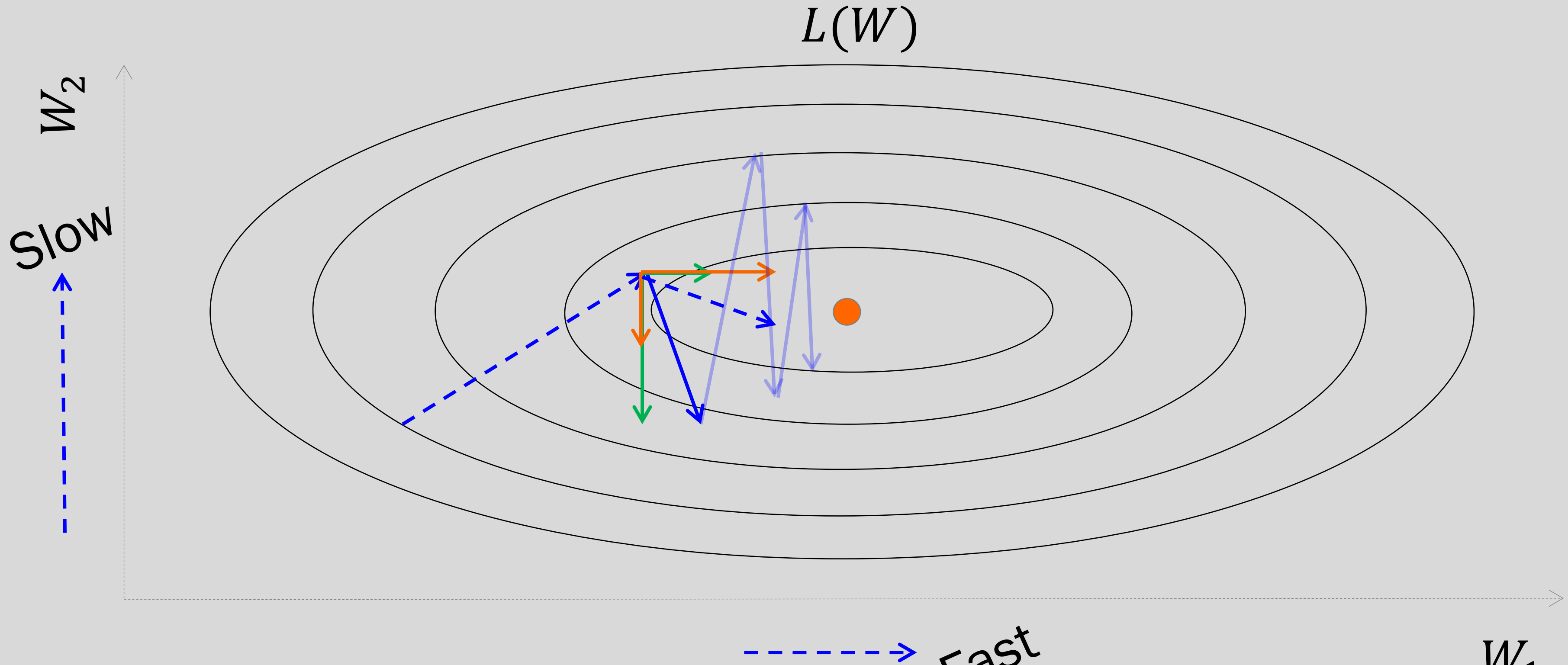


Oscillations along vertical direction

- Learning must be slower along parameter 2

Use a different learning rate for each parameter?

Adaptive Learning Rates

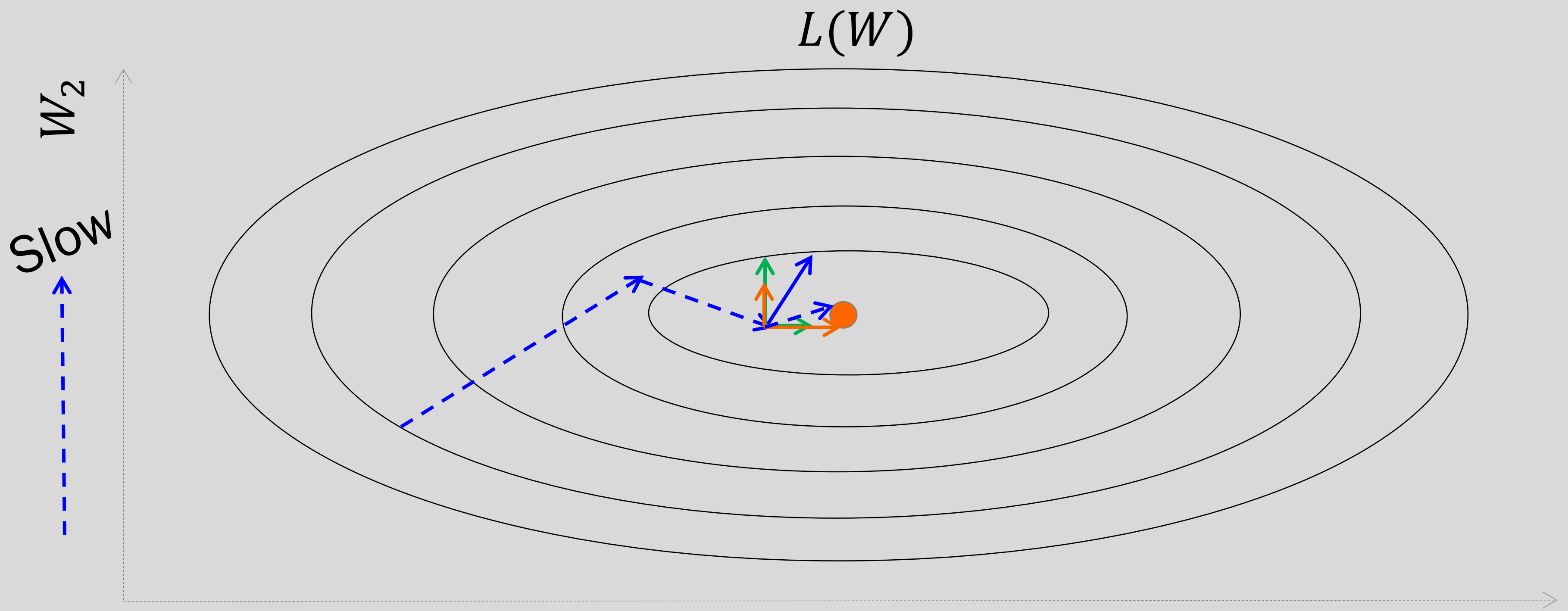


Oscillations along vertical direction

- Learning must be slower along parameter 2

Use a different learning rate for each parameter?

Adaptive Learning Rates



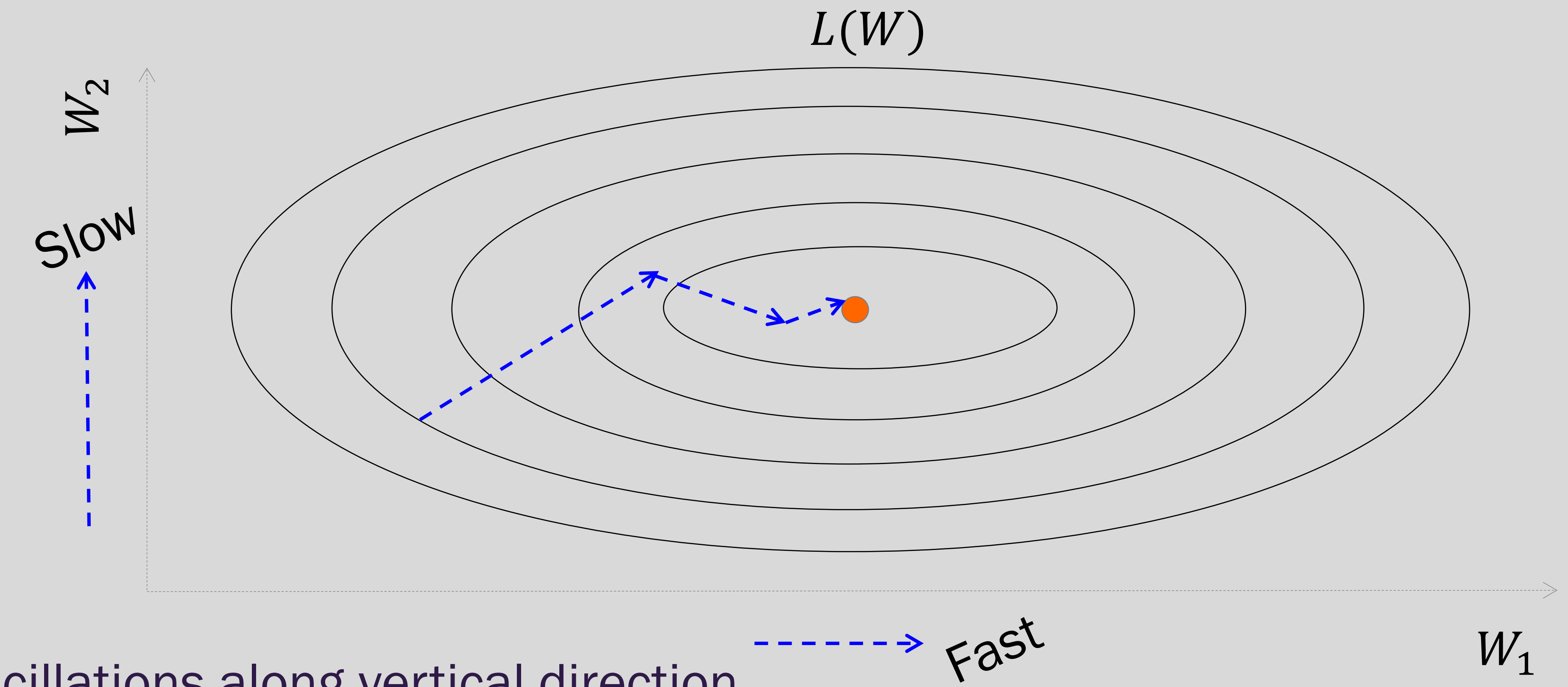
Oscillations along vertical direction

- Learning must be slower along parameter 2

Use a different learning rate for each parameter?

School of
COMPUTING
Clemson University

Adaptive Learning Rates



Oscillations along vertical direction

- Learning must be slower along parameter 2

Use a different learning rate for each parameter?

AdaGrad

- Accumulate squared gradients:

$$r_i = r_i + g_i^2$$

g is the gradient

- Update each parameter:

$$W_i = W_i - \frac{\epsilon}{\delta + \sqrt{r_i}} g_i$$

Inversely proportional to cumulative squared gradient

- Greater progress along gently sloped directions

AdaGrad

δ is a small number, making sure this does not become too large

Old gradient descent:

$$g = \frac{1}{m}\sum_i \nabla_W L(f(x_i;W), y_i) \qquad W^* \quad W - \lambda g$$

We would like $\lambda's$ not to be the same and inversely proportional to the $|g_i|$

$$W_i^* = W_i - \lambda_i g_i \qquad \lambda_i \propto \frac{1}{|g_i|} = \frac{1}{\delta + |g_i|}$$

New gradient descent with adaptive learning rate:

$$r_i^* = r_i + g_i^2 \qquad W_i^* = W_i - \frac{\epsilon}{\delta + \sqrt{r_i}} \mathrm{g_i}$$

RMSProp

- For non-convex problems, AdaGrad can prematurely decrease learning rate
- Use exponentially weighted average for gradient accumulation

$$r_i = \rho r_i + (1-\rho) g_i^2$$

$$W_i = W_i - \frac{\epsilon}{\delta + \sqrt{r_i}} g_i$$

Adam

- RMSProp + Momentum
- Estimate first moment:

$$v_i = \rho_1 v_i + (1-\rho_1) g_i$$

- Estimate second moment:

$$r_i = \rho_2 r_i + (1-\rho_2) g_i^2$$

- Update parameters:

$$W_i = W_i - \frac{\epsilon}{\delta + \sqrt{r_i}} \nu_i$$

Also applies bias correction to v and r

Works well in practice, is fairly robust to hyper-parameters

School of COMPUTING Clemson University